

RideSmart: Modelagem e Análise de Rotas Urbanas com Grafos na Cidade de Natal-RN

José Alex Araújo de Santana

Departamento de Engenharia da Computação e Automação (DCA)
Universidade Federal do Rio Grande do Norte, Natal, Brasil

Resumo—O projeto RideSmart modela o problema de seleção do melhor ponto de embarque em mobilidade urbana como caminho mínimo em grafos. Sobre a rede viária real de Natal-RN (18.576 nós, 48.180 arestas), obtida via OSMnx, foram implementados e comparados quatro algoritmos — Dijkstra Simples, Dijkstra com Heap, A* geográfico e Bellman-Ford — em cinco cenários com pesos de distância, tempo e trânsito sintético. O caso de estudo considera como origem a Rua Jaguarari e como destino o Natal Shopping, ambos em Natal-RN. Foram identificados 170 candidatos ao ponto de embarque P dentro do raio de 500 m. O A* expandiu apenas 356 nós contra 1.544 do Dijkstra Simples, com tempo de 5,00 ms vs. 5.143,33 ms, confirmado por benchmark de 20 execuções (D.P. de 0,68 ms). O trânsito sintético elevou o custo temporal em 50,2% sem alterar a rota escolhida. O ponto de embarque ótimo (nó 8644620966, a 192 m de A) reduziu a distância viária de 2.624 m para 2.365 m — uma economia de 259 m em relação à rota direta —, demonstrando que a caminhada táctica gera ganho real de distância, embora não de tempo neste cenário específico.

Index Terms—Grafos, Caminho Mínimo, Dijkstra, A*, Bellman-Ford, Mobilidade Urbana, OSMnx, Natal-RN

I. INTRODUÇÃO

Aplicativos de mobilidade como Uber e 99 selecionam rotas e pontos de embarque automaticamente, mas raramente expõem seus critérios ao usuário. O RideSmart investiga esse problema: dado um ponto de origem A , um destino B e um raio máximo de caminhada X , qual ponto de embarque P minimiza o custo total da viagem $A \rightarrow P$ (a pé) $+ P \rightarrow B$ (carro)?

A rede viária de Natal-RN foi escolhida por representar uma cidade de médio porte com malha heterogênea. O caso de estudo parte da Rua Jaguarari, uma via residencial de conectividade limitada, em direção ao Natal Shopping. Essa escolha de origem é metodologicamente relevante porque força o algoritmo a explorar um espaço de candidatos P muito maior (170 nós vs. cenários anteriores com 3 nós), evidenciando com mais clareza o comportamento dos algoritmos e a utilidade de caminhar até um ponto de embarque alternativo.

Os objetivos são: (i) modelar a rede como dígrafo ponderado; (ii) implementar e comparar quatro algoritmos de caminho mínimo; (iii) analisar o impacto do trânsito sintético; e (iv) avaliar o *trade-off* entre caminhar até um ponto alternativo e partir diretamente de A .

II. MODELAGEM DO PROBLEMA

A rede viária é representada como dígrafo ponderado $G = (V, E, w)$: nós (V) são cruzamentos; arestas (E) são

segmentos de via com sentido; pesos (w) codificam os critérios de otimização. O dígrafo é necessário para respeitar vias de mão única. Os dados foram obtidos via OSMnx [1], resultando em 18.576 nós e 48.180 arestas.

A. Funções de Custo

Três funções de custo foram definidas (Tabela I). O peso $\text{travel_time} = \text{length} / \text{speed_kph}$; o peso $\text{travel_time_traffic}$ multiplica travel_time por fatores por tipo de via (Tabela II), simulando hora de pico.

Tabela I
FUNÇÕES DE CUSTO

Cenário	Peso	Unidade	Fator (ex.)
C1 — Distância	length	m	—
C2 — Tempo	travel_time	s	—
C3 — Trânsito	$\text{travel_time_traffic}$	s	$\times 1,0\text{--}2,5$

Tabela II
FATORES DE CONGESTIONAMENTO POR TIPO DE VIA

Via	Fator	Via	Fator
motorway	$\times 2,5$	tertiary	$\times 1,4$
trunk	$\times 2,2$	residential	$\times 1,1$
primary	$\times 2,0$	unclassified	$\times 1,2$
secondary	$\times 1,7$	service/living	$\times 1,0$

B. Função de Custo Multimodal

A combinação dos trechos de caminhada e de carro exige que ambos os custos sejam expressos na *mesma unidade*. Adotou-se o tempo em segundos como métrica unificadora, de modo que o custo total de uma rota com ponto de embarque P é definido por:

$$C_{\text{total}}(P) = T_{\text{walk}}(A, P) + T_{\text{drive}}(P, B), \quad (1)$$

onde $T_{\text{walk}}(A, P)$ é o tempo de caminhada de A até P e $T_{\text{drive}}(P, B)$ é o tempo de deslocamento de carro de P até B .

A conversão da distância em metros para tempo de caminhada emprega uma velocidade média padrão de $v_{\text{walk}} = 1,39$ m/s (5 km/h), valor amplamente adotado na literatura de planejamento urbano [6], de forma que:

$$T_{\text{walk}}(A, P) = \frac{d(A, P)}{v_{\text{walk}}} = \frac{d(A, P)}{1,39}, \quad (2)$$

com $d(A, P)$ em metros e T_{walk} resultante em segundos. Essa normalização garante que a soma na Eq. (1) seja justa, evitando comparações entre grandezas de escalas distintas.

C. Pontos Experimentais e Candidatos P

Os pontos experimentais estão na Tabela III. O raio máximo de caminhada foi fixado em $X = 500$ m, resultando em 170 candidatos P . A Tabela IV lista os dez mais próximos de A .

Tabela III
PONTOS DE ORIGEM E DESTINO

Ponto	Local	Coord.
A (Origem)	Rua Jaguarari	−5,8289, −35,2219
B (Destino)	Natal Shopping	−5,8418, −35,2099
Dist. linha reta A→B: 1.924 m Rota viária C4: 2.624 m		

Tabela IV
TOP 10 CANDIDATOS AO PONTO DE EMBARQUE P (≤ 500 M DE A)

Nó (P)	Dist. A→ P	T. Caminhada
506566849	80 m	57 s
506566862	84 m	60 s
505488924	96 m	69 s
505487929	105 m	76 s
506566850	121 m	87 s
506566866	121 m	87 s
506566851	128 m	92 s
505488613	133 m	96 s
1437116037	139 m	100 s
506566848	146 m	105 s

III. ALGORITMOS UTILIZADOS

Dijkstra Simples [2]: busca linear do nó de menor custo a cada iteração; complexidade $\mathcal{O}(V^2)$. Simples de implementar, mas inviável em grafos grandes.

Dijkstra com Heap: substitui a busca linear por min-heap, reduzindo a complexidade para $\mathcal{O}((V + E) \log V)$. Usa *lazy deletion*: entradas desatualizadas são ignoradas na extração.

A* Geográfico [3]: estende o Dijkstra com Heap pela função $f(n) = g(n) + h(n)$, onde $g(n)$ é o custo acumulado do início até n e $h(n)$ é a *heurística* que estima o custo restante de n até o destino B .

A função heurística adotada foi a **distância de Haversine**:

$$h(n) = 2R \arcsin \left(\sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_n \cos \phi_B \sin^2 \frac{\Delta\lambda}{2}} \right), \quad (3)$$

onde $R \approx 6.371$ km é o raio médio da Terra, ϕ denota latitudes e λ longitudes em radianos. A heurística é **admissível** porque a distância em linha reta (Haversine) entre dois pontos na superfície terrestre nunca *superestima* o custo real do caminho viário — todo trecho de rua, por mais curto, deve ser pelo menos tão longo quanto a linha reta correspondente.

Formalmente, $h(n) \leq h^*(n)$ para todo nó n , onde $h^*(n)$ é o custo ótimo real de n até B .

Além de admissível, a heurística é **consistente** (ou monótona): para qualquer aresta (n, m) com custo $c(n, m)$, vale $h(n) \leq c(n, m) + h(m)$, o que garante que cada nó seja expandido no máximo uma vez e que o A* se comporte como uma busca de custo uniforme dirigida ao destino. Para cenários de tempo (C2 e C3), a Haversine é dividida pela velocidade máxima permitida no grafo (33,33 m/s $\approx$ 120 km/h), mantendo a admissibilidade para o peso temporal.

Bellman-Ford [4]: realiza $|V| - 1$ relaxamentos sobre todas as arestas; complexidade $\mathcal{O}(V \cdot E)$. Suporta arestas de peso negativo e detecta ciclos negativos — vantagens irrelevantes em redes viárias, mas relevante como algoritmo de referência da literatura.

Tabela V
COMPLEXIDADE DOS ALGORITMOS

Algoritmo	Complexidade	Estrutura
Dijkstra Simples	$\mathcal{O}(V^2)$	Lista linear
Dijkstra Heap	$\mathcal{O}((V + E) \log V)$	Min-heap
A*	$\mathcal{O}(E \log V)$	Min-heap + heurística
Bellman-Ford	$\mathcal{O}(V \cdot E)$	Relaxamento iterativo

IV. EXPERIMENTOS E RESULTADOS

Os experimentos cobriram cinco cenários: C1 (distância), C2 (tempo), C3 (trânsito), C4 (rota direta A→B, caso-base) e C5 (ganho ao caminhar). Para C1–C3, os 170 candidatos P foram avaliados com os quatro algoritmos, exibindo-se os 10 melhores; para C4, apenas a rota direta. A Tabela VI compara o desempenho no cenário C4.

Tabela VI
CENÁRIO C4 — ROTA DIRETA A→B (PESO: DISTÂNCIA)

Algoritmo	Custo (m)	Nós Exp.	Tempo (ms)
Dijkstra Simples	2.624	1.544	5.143,33
Dijkstra Heap	2.624	1.545	11,29
A*	2.624	356	5,00
Bellman-Ford	2.624	111	3.947,51

Todos convergiram para 2.624 m, confirmando a correção das implementações. Para verificar a reprodutibilidade, cada algoritmo foi executado 20 vezes consecutivas (Tabela VII). Desvios padrão pequenos indicam algoritmos estáveis; desvios grandes revelam sensibilidade ao estado do sistema.

Tabela VII
BENCHMARK DE ROBUSTEZ — 20 EXECUÇÕES (CENÁRIO C4)

Algoritmo	Média (ms)	D.P. (ms)	Med. (ms)	Mín. (ms)	Máx. (ms)
Dijkstra Simples	4.356,54	483,89	4.138,40	3.862,01	5.157,23
Dijkstra Heap	9,80	1,08	9,35	9,05	13,71
A*	4,54	0,68	4,27	4,19	6,90
Bellman-Ford	3.960,19	364,21	3.833,68	3.470,62	4.594,24

A Tabela VIII resume o melhor P por cenário e a Tabela IX o ganho ao caminhar para os 20 candidatos avaliados.

Tabela VIII
MELHOR P POR CENÁRIO (DIJKSTRA HEAP)

Cenário	P Ótimo	Caminhada	Custo
C1 — Distância	8644620966	192 m	2.365,1 m
C2 — Tempo	8644620966	192 m	209,7 s
C3 — Trânsito	8644620966	192 m	315,0 s

Tabela IX
CENÁRIO C5 — GANHO AO CAMINHAR (BASE: 349,0 s / 2.624 M DIRETO)

P	Walk	T. Walk	T. Carro	Ganho (s)
506566849	80 m	57 s	359 s	-68
506566862	84 m	60 s	338 s	-49
505488924	96 m	69 s	337 s	-57
505487929	105 m	76 s	336 s	-62
506566850	121 m	87 s	350 s	-88
506566866	121 m	87 s	365 s	-103
506566851	128 m	92 s	349 s	-92
505488613	133 m	96 s	325 s	-72
1437116037	139 m	100 s	324 s	-75
506566848	146 m	105 s	375 s	-131

V. DISCUSSÃO CRÍTICA

A. Eficiência Computacional

O A^* expandiu 76,9% menos nós que o Dijkstra Simples (356 vs. 1.544) e foi $1.029\times$ mais rápido (5,00 ms vs. 5.143,33 ms), com D.P. de apenas 0,68 ms — evidência de comportamento estável, reflexo da heurística geográfica (ver Seção III). A diferença em nós expandidos é mais pronunciada neste cenário do que em experimentos anteriores (172 vs. 771), o que se explica pelo maior distância linha reta $A \rightarrow B$ (1.924 m), que exige explorar uma região maior do grafo antes de atingir o destino. O Dijkstra Heap, com 11,29 ms e D.P. de 1,08 ms, demonstra que a estrutura de dados é o fator dominante sobre o Dijkstra Simples (D.P. de 483,89 ms), cuja instabilidade cresce com o espaço explorado.

O Bellman-Ford, embora expanda apenas 111 iterações de relaxamento, consumiu 3.947,51 ms — resultado direto de sua complexidade de pior caso $\mathcal{O}(V \cdot E)$. Para a rede de Natal-RN, com $|V| = 18.576$ nós e $|E| = 48.180$ arestas, isso implica até $18.576 \times 48.180 \approx 8,9 \times 10^8$ operações de relaxamento no pior caso. Em contraste, o Dijkstra com Min-Heap opera em $\mathcal{O}((V + E) \log V)$; para os mesmos valores, $(18.576 + 48.180) \times \log_2(18.576) \approx 4,4 \times 10^6$ operações — uma diferença de mais de duas ordens de grandeza. Embora o Bellman-Ford apresente a vantagem teórica de suportar arestas com pesos negativos e detectar ciclos negativos, tais propriedades são irrelevantes em redes viárias — cujos pesos (distância e tempo) são estritamente positivos —, tornando-o *inadequado para aplicações de produção* em grafos viários de médio e grande porte.

B. Distância versus Tempo

Os três cenários elegeram o mesmo P ótimo (nó 8644620966, a 192 m de A). Em distância, a caminhada até esse ponto reduziu o percurso de carro de 2.624 m para 2.365,1 m — uma economia de 259 m ($\approx 9,9\%$) em relação à rota direta $C4$. Em tempo, o custo foi de 209,7 s ($C2$) e 315,0 s ($C3$). Isso evidencia que menor distância não implica menor tempo, pois o tipo e a velocidade da via são determinantes.

C. Trânsito Sintético

O custo temporal do melhor P saltou de 209,7 s ($C2$) para 315,0 s ($C3$), aumento de 50,2%, reflexo dos fatores de congestionamento nas vias ao longo da rota. A rota selecionada permaneceu a mesma, mas o impacto temporal demonstra que a modelagem de congestionamento é crítica para a experiência do usuário.

D. Trade-off Caminhar / Tempo e Quando Caminhar Ajuda

Neste cenário, o resultado do $C5$ é mais rico do que em experimentos anteriores. O melhor P em distância (nó 8644620966, 192 m de caminhada) reduziu a rota de carro em 259 m, mas o tempo total de viagem ainda foi maior do que o direto: o pedestre caminha 138 s e o carro percorre 315 s, totalizando 453 s frente aos 349 s da rota direta. Todos os candidatos P resultaram em ganho de tempo negativo (Tabela IX), com o melhor caso perdendo apenas 49 s (nó 506566862, 84 m de caminhada).

Caminhar atrapalha em tempo neste cenário porque: (i) a origem na Rua Jaguarari, embora de conectividade mais limitada que uma parada de ônibus, ainda possui acesso razoável à malha viária secundária; (ii) a distância $A \rightarrow B$ de 1,92 km, embora maior que nos experimentos anteriores, não é suficientemente longa para que o ganho de velocidade nos trechos de carro compense o tempo de caminhada adicionado; e (iii) nenhum candidato P oferece acesso a uma via expressamente mais rápida que justifique o custo de deslocamento a pé. Contudo, a redução de 259 m em distância viária evidencia que caminhar até P pode ser vantajoso em cenários onde o critério de otimização é distância — por exemplo, para reduzir o custo da corrida em aplicativos tarifados por quilômetro.

VI. CONCLUSÃO

O RideSmart demonstrou a viabilidade de modelar a seleção de ponto de embarque como caminho mínimo em dígrafo ponderado real, com o caso de estudo da Rua Jaguarari ao Natal Shopping em Natal-RN. O espaço de 170 candidatos P permitiu uma análise mais abrangente do que cenários anteriores. O A^* superou os demais em eficiência (5,00 ms, D.P. 0,68 ms, 76,9% menos nós que o Dijkstra Simples); o Dijkstra Heap oferece equilíbrio prático (11,29 ms); o Bellman-Ford, embora correto, é inadequado para grafos viários positivos em produção, dado o custo assintótico $\mathcal{O}(V \cdot E)$, que neste grafo equivale a $\sim 8,9 \times 10^8$ operações no pior caso. O trânsito sintético impactou 50,2% o custo temporal sem mudar a rota; a caminhada até o melhor P gerou ganho real de distância (259 m) mas não de tempo neste cenário.

A. Limitação: Ausência do Tempo de Dispatch

Um aspecto crítico ausente no modelo atual é o **tempo de dispatch**: assume-se implicitamente que o veículo já se encontra estacionado no ponto de embarque P no momento em que o pedestre chega. Em um aplicativo real de mobilidade, o algoritmo deve incorporar a coordenada dinâmica do motorista C e o tempo de deslocamento $T_{\text{drive}}(C, P)$ até o ponto escolhido, pois um ponto P ótimo para o pedestre pode exigir um longo desvio para o motorista, aumentando o tempo total da operação. Formalmente, o custo completo seria:

$$C_{\text{total}}^*(P) = T_{\text{walk}}(A, P) + T_{\text{drive}}(C, P) + T_{\text{drive}}(P, B), \quad (4)$$

onde $T_{\text{drive}}(C, P)$ é o tempo de deslocamento do motorista até P . A omissão desse termo pode levar à seleção de um ponto de embarque que é teoricamente ótimo para o pedestre, mas que penaliza o motorista — e, conseqüentemente, o tempo total do serviço.

REFERÊNCIAS

- [1] G. Boeing, “OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks,” *Comput. Environ. Urban Syst.*, vol. 65, pp. 126–139, 2017.
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numer. Math.*, vol. 1, pp. 269–271, 1959.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
- [4] R. Bellman, “On a routing problem,” *Q. Appl. Math.*, vol. 16, pp. 87–90, 1958.
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [6] K. Eakin and A. Conner, “Pedestrian travel speed and accessibility,” *Transportation Research Record*, vol. 1828, pp. 101–107, 2003.